# Report 1: Q9
# Matrix Chain Multiplication (DP vs Greedy)

**Title**

Comparative Analysis of Matrix Chain Multiplication Using Dynamic Programming and Greedy Approach

**Aim**

To compare Dynamic Programming and Greedy strategies for solving the Matrix Chain Multiplication problem and evaluate their efficiency, correctness, and optimization capabilities.

**Problem Statement**

Given a sequence of matrices:

$A_1 = 10 \times 20$

$A_2 = 20 \times 30$

$A_3 = 30 \times 40$

Determine the optimal order of multiplication that minimizes the total number of scalar multiplications.

**Introduction**

Matrix Chain Multiplication (MCM) is a classical optimization problem in Dynamic Programming. Since matrix multiplication is associative, matrices can be multiplied in different orders. Although the final result remains the same, the computational cost differs significantly based on the multiplication order.

The objective is to find the multiplication sequence that minimizes scalar multiplication operations.

**Dynamic Programming Approach**

Dynamic Programming computes the minimum cost by evaluating all possible parenthesizations and storing intermediate results.

For the given matrices:

**Option 1: $(A_1A_2)A_3$**

Cost of $A_1A_2$:

= 10 × 20 × 30

= 6000

Resultant matrix:

10 × 30

Cost of multiplying with $A_3$:

= 10 × 30 × 40

= 12000

Total Cost:

= 6000 + 12000

= 18000

**Option 2: $A_1(A_2A_3)$**

Cost of $A_2A_3$:

= 20 × 30 × 40

$= 24000$

Resultant matrix:

$20 \times 40$

Cost of multiplying with $A_1$:

$= 10 \times 20 \times 40$

$= 8000$

Total Cost:

$= 24000 + 8000$

$= 32000$

**Optimal Cost**

Minimum Cost = 18000

Optimal Order = $(A_1A_2)A_3$

**Greedy Approach**

The Greedy strategy selects the multiplication having the minimum immediate cost.

Immediate multiplication costs:

- $A_1A_2 = 6000$
- $A_2A_3 = 24000$

Since 6000 is smaller, Greedy selects $(A_1A_2)$ first.

Remaining multiplication cost:

$10 \times 30 \times 40 = 12000$

Total Cost:

$6000 + 12000 = 18000$

**Comparison of Results**

| **Method** | **Cost** |
|---|---|
| Dynamic Programming | 18000 |
| Greedy | 18000 |

For this specific example, both methods produce the same result.

**Why Greedy Fails**

Greedy decisions are based only on local optimization and do not consider future consequences.

Example:

Matrices:

- $10 \times 100$
- $100 \times 5$
- $5 \times 50$

A locally optimal choice may not lead to the globally optimal solution.

Dynamic Programming evaluates all possibilities and always guarantees the minimum cost.

**Time Complexity Analysis**

| **Method** | **Time Complexity** | **Space Complexity** |
|---|---|---|
| Greedy | $O(n)$ | $O(1)$ |
| Dynamic Programming | $O(n^3)$ | $O(n^2)$ |

**Advantages of Dynamic Programming**

1. Guarantees optimal solution.
2. Avoids repeated calculations.
3. Suitable for large matrix chains.
4. Produces globally optimal parenthesization.

**Limitations**

1. Requires additional memory.
2. Higher computational complexity.
3. Implementation is more complex than Greedy.

**Applications**

1. Scientific Computing
2. Machine Learning
3. Graphics Processing
4. Numerical Simulations
5. Big Data Analytics

**Conclusion**

Matrix Chain Multiplication demonstrates the importance of optimization in computational problems. While the Greedy approach may occasionally produce optimal results, it cannot guarantee correctness for all cases. Dynamic Programming systematically explores all possible multiplication orders and always produces the optimal solution. Therefore, Dynamic Programming is the preferred approach for solving Matrix Chain Multiplication problems.